

6교시 · Multi-Agent — Deep Research

5교시까지 쓰던 `agent-lab` 을 `run6.bat` 으로 다시 쓴다.

지금까지는 Agent 하나가 다 하고 있었다

도구를 받고, 질문을 전부 받고, 답도 혼자 쓰는 **실무자 한 명** 구조. AI가 한 번에 읽고 처리할 수 있는 분량은 정해져 있음 — 도구 설명문, 가져온 자료, 실행 결과가 전부 그 안에 들어간다. **대화가 길어지면 앞부분을 요약** 하게 되고, 요약된 부분의 디테일은 사라진다. 도구가 수십 개, 조사 자료가 쌓이면 그 한도가 금방 참.

왜 나누는가

만들 것: Deep Research의 강화 버전

일반 Deep Research(조사→집계→분석→결론)에 **연구 방법론 수립, 시뮬레이션 코드 작성·실행, 결과 정리·분석**을 더한 버전.

일이 성격이 다른 두 덩어리로 갈림:

- **자료 조사** — 어떤 방법이 있는지 찾고 이 문제에 무엇을 쓸지 결정
- **코드 작성과 실행** — 실제로 돌려서 수치를 냄

지금은 둘 다 가벼워서 한 명이 다 해도 되지만, 조사 쪽에 사내 문서 저장소와 제약 조건이 붙고 실행 쪽에 코딩 규칙과 DB 접속이 붙으면 **각각 따로 무거워진다** — 그때 한 명에게 다 맡기면 문제가 생김.

셋으로 나누기

- **연구책임자** — 무엇을 할지 정하고 맡긴다. 직접 조사·실행하지 않음
- **자료조사 연구원** — 분석 방법을 설계, 웹 검색 등 수행
- **실험수행 연구원** — 실험 계획 수립·수행, Python 코드 작성 및 실행

규칙: 두 연구원은 서로를 부를 수단이 없다. 모든 전달은 연구책임자를 거친다. **최종 수치는 실행된 코드의 결과만 사용한다.**

나누는 대가와 이득

- **비용(통신 비용)**: 위임마다 지시문 작성→전달→응답 대기. 전달 과정에서 맥락이 깎임(연구책임자가 지시문에 담지 않은 건 연구원이 모름) — 실습에서 연구 하나에 몇 분 걸리는 이유의 상당 부분.
- **이득(유지보수 용이성)**: 에이전트 하나가 감당하는 지시/제약이 줄어들어 어느 문장이 어떤 행동을 만들었는지 짚기 쉬움, 쥔 도구가 적으면 오작동 원인 파악도 쉬움, 문제 시 들여다볼 범위가 시스템 전체→에이전트 하나로 좁혀짐.
- **부가 이득(확장성)**: 단일 거대 에이전트는 새 기능 추가 시 길어진 지시문/도구목록이 얹혀 기존 동작이 흔들림(4교시에서 설명문 한 줄로 선택이 뒤집힌 사례). 역할 분리 구조에서는 기존 연구원은 그대로 두고 **연구책임자가 맡길 상대를 하나 늘리기만** 하면 됨.

실습 — 구동

`run6.bat` 실행 → [에이전트] 탭 등장. 프로세스 카드에서 자료조사 연구원·실험수행 연구원 각각 [기동] → PID·포트 확인 (3개 프로세스가 각자 떠 있고 그중 하나가 나머지에 일을 맡기는 구조).

도구 분리 확인: 세 줄의 도구 항목을 나란히 읽어보면 5교시에 붙인 서버 도구가 어느 연구원에게 갔는지 보인다. 4교시는 역할을 문구로 막아 표현 우회 가능성이 있었지만, 여기서는 **도구 자체를 나눠 줌** — 없는 도구는 애초에 부를 수 없음(더 확실한 방어).

연구 맡기기 — 동작 확인

동전을 10번 던져 앞면이 6번 이상 나올 확률을 구하려고 합니다.
어떤 방법으로 구할 수 있는지 조사하고, 파이썬으로 실행해 확인하세요.
정량 결과는 실제 코드 실행 결과를 사용하세요.

본 실습 — 라인 전환 수익성 시뮬레이션

데이터: `data/research/line_transition.csv` (교육용 가상 라인/수치). F-3 라인을 차세대 공정으로 전환 시 수익이 낮게 시작해 몇 달에 걸쳐 성숙 수출까지 오른다는 가정.

F-3 라인을 차세대 공정으로 전환하려고 합니다.
전환 직후에는 수익이 낮고, 몇 달에 걸쳐 성숙 수출까지 올라간다고 가정합니다.

제공된 사업 가정(`line_transition.csv`)을 바탕으로 적절한 분석 방법을 조사하고,
시뮬레이션을 설계하여 실제 파이썬 코드로 실행하세요.
24개월 동안의 누적 영업이익 분포와 적자 발생 확률을 분석하고,
결과에 가장 큰 영향을 미치는 변수를 찾아주세요.

정량 결과는 실제 코드 실행 결과를 사용하세요.

몇 분 소요(조사 담당이 문서 읽고, 실험 담당이 코드를 고쳐가며 실행하는 시간) → **Agent 간 주고받은 정보 로그**가 쌓임(파란 상자=맡긴 것, 회색 상자=돌려준 것).

로그에서 확인할 것

- **자료조사 연구원**이 돌려준 상자: 어떤 도구 사용, 무엇을 읽었는지, 무엇을 설계했는지.
`line_transition.csv` 는 변수마다 제공 형태가 다름(최소·기준·최대 / 평균·표준편차 / 값 하나 / 수치가 아니라 "방식"을 적은 행도 있음) — 조사 담당이 이 차이를 어떻게 반영했는지 대조.
- **실험수행 연구원**이 돌려준 상자: 파일을 읽은 흔적 → 실행한 코드 → 코드가 찍은 출력. 최종 결론 수치(중앙값, 하위 5%, 상위 5%, 적자 발생 확률, 민감도 1위 변수)를 기록.
- **연구책임자** → **실험수행 연구원** 지시문: 무엇을 적어 보냈는지 확인. **핵심:** 두 연구원은 서로의 결과를 모르므로, 앞 단계 결과가 다음 지시문에 실려 넘어가야 협업이 성립한다.
- **역할 준수 확인:** 연구책임자가 직접 계산/수치를 지어내지 않았는가, 자료조사가 수치·코드를 만들려 하지 않았는가, 실험수행이 판단(전환 추진 여부 등)을 붙이지 않았는가.

이어서 물어보기 (컨텍스트 유지)

연구책임자는 방금 연구를 기억한다 → 조건 변경 후속 질의 가능:

공정 안정화가 예상보다 늦어져 수율 램프 기간이
최소 8개월, 기존 14개월, 최대 20개월이라면 어떻게 되나?

나머지 조건과 확률변수를 뽑는 방식은 그대로 두고 다시 계산해 줘.

이번엔 수치보다 **연구책임자의 판단**(누구에게 무엇을 다시 말기는지)이 관찰 포인트. **[새 연구로 시작]**을 누르면 기억이 지워져 **처음부터 재조사** — 후속 질의 실습 중에는 누르지 말 것.

정리 — 나눌지 말지 판단 기준

- 다루는 자료가 서로 다른가 (조사 자료 vs 실행 자료가 겹치지 않으면 한 곳에 쌓아둘 이유 없음)
- 필요한 지시·도구·권한이 다른가
- 만든 쪽과 검토하는 쪽을 갈라야 하는가 (스스로 짠 걸 스스로 검토하면 놓침 — 실무에서 가장 흔한 분리 이유)

핵심: 오늘 셋으로 나눈 건 "셋이 더 나아서"가 아니라 조사와 실행이 각각 다른 도구·다른 지시가 필요했기 때문. **나눌 이유가 없으면 한 명이 낫다.** 중요한 건 에이전트 수가 아니라 **왜 책임을 나눴는가.**

- 연구책임자는 말기고 받고 다음을 정한다 — 직접 하지 않는다
- 앞 단계 결과가 **다음 지시문에 실려** 넘어가는 것이 협업의 실체
- 역할의 경계는 두 겹: **지시문으로** 정하고 **도구로** 막는다
- 설계는 한 번 해두면 값을 바꿔 재사용 가능(조사를 다시 안 해도 됨)
- **결과를 읽는 법:** 중앙값 하나가 아니라 하위 5%·상위 5%·적자확률까지 — "하나의 예상값" vs "분포를 본다"의 차이. 남길 것은 숫자가 아니라 **분포를 본다는 감각.**

지금 가진 것: **agent-lab** — 도구 3개 + 서버 2개 + 연구원 2명

이후 7~8교시는 오늘 만든 것을 바탕으로 **각자의** 캡스톤 프로젝트를 만든다.